

Оптимизация запросов

Общее назначение [оптимизации запросов](#) — повышение производительности приложения путем изменения:

- структуры БД
- запросов к БД

с целью уменьшения использования вычислительных ресурсов при выполнении запроса.

Структура БД

Организация структуры БД, описание таблиц, отношений между ними, организация первичных ключей и создание индексов подробно рассмотрены [здесь](#).

Запросы к БД

Нередко возникают ситуации, когда запрос работает долго, потребляя значительные ресурсы памяти и дисков. Назовем такие запросы неэффективными или ресурсоемкими.

Проанализировать запрос помогает Explain, работа с которым подробно описана в разделе документации "[Контроль использования индексов](#)".

После анализа плана выполнения запроса осуществляется его оптимизация. Оптимизация запроса предполагает удаление причин неэффективности запроса, среди которых наиболее весомыми являются:

- проблемы с [индексами](#);
- проблемы с [хинтами](#);
- неэффективно построенный [запрос](#).

В документации достаточно подробно описана работа с индексами и с запросами. К изложенным материалам можно добавить общие принципы оптимизации:

1. Минимизация количества запросов.

Один "большой" SQL запрос лучше серии "маленьких".

2. Группировка нескольких запросов в единую транзакцию для уменьшения нагрузки при выполнении завершения транзакции.

Меньше операций записи на диск. Однако следует учесть, что "длинные" транзакции — не всегда лучший выбор для web-приложений.

3. Минимизация объемов данных выборки (включая промежуточные).

- При написании запросов не использовать выборку всех полей:

НЕТ:

```
SELECT * FROM table;
```

ДА:

```
SELECT t1, t2, t3, t4 FROM table;
```

Перечислять только нужные поля — это сократит количество выбираемых и пересылаемых данных; даже если необходимы все поля в таблице, лучше их перечислить:

- это повышает читабельность кода, при использовании звездочки невозможно узнать, какие поля есть в таблице без заглядывания в нее;
- со временем количество столбцов в таблице может изменяться, и если сегодня это пять INT столбцов, то через месяц могут добавиться TEXT и BLOB поля, которые будут замедлять выборку.
- использовать WHERE условия
- ограничивать результат запроса использованием LIMIT

4. Оптимизация запросов, для которых нельзя заставить оптимизатор использовать индексы, и которые будут всегда вызывать полный просмотр таблицы:

- Запрос такого вида не будет использовать индекс по полю field

```
SELECT max(field)/min(field) FROM big_table
```

возможное решение:

```
SELECT field FROM big_table ORDER BY field DESC LIMIT 1;
```

- Минимизация медленных [count\(*\)](#)

```
SELECT count(*) FROM big_table;
```

5. Минимизация количества операций DELETE.

Возможно использование [TRUNCATE TABLE](#)

6. Массивные операции вместо группы row-level операций.

- один UPDATE вместо цикла UPDATE-ов
- COPY вместо группы INSERT-ов

7. Подготовленные запросы.

У подготовленных запросов есть два главных преимущества:

- Запрос необходимо однажды подготовить и затем его можно запускать требуемое количество раз, как с теми же, так и с отличающимися параметрами.

Когда запрос подготовлен, СУБД анализирует его, компилирует и оптимизирует план его выполнения. В случае сложных запросов этот процесс может занимать ощутимое время и заметно замедлить работу приложения, если потребуется много раз выполнять запрос с разными параметрами. При использовании подготовленного запроса СУБД анализирует/компилирует/оптимизирует запрос любой сложности только один раз, а приложение запускает на выполнение уже подготовленный шаблон. Таким образом подготовленные запросы потребляют меньше ресурсов и работают быстрее.

- Параметры подготовленного запроса не требуется экранировать кавычками. Если в приложении используются исключительно подготовленные запросы, разработчик может быть уверен, что никаких SQL-инъекций случиться не может (однако, если другие части текста запроса записаны с неэкранированными символами, SQL инъекции все же возможны; здесь речь идет именно о параметрах).

Пример 1:

```
1 PREPARE tableplan (int, text, bool, numeric) AS
2 INSERT INTO table VALUES($1, $2, $3, $4);
3 EXECUTE tableplan(1, 'Hunter Valley', 't', 200.00);
```

Пример 2:

```
1 PREPARE usrrptplan (int, date) AS
2 SELECT * FROM users u, logs l WHERE u.usrid=$1 AND u.usrid=l.usrid
   AND l.date = $2;
3 EXECUTE usrrptplan(1, current_date);
```

Рекомендуемая документация

1. Боровиков К. ["PostgreSQL для разработчика. Оптимизация запросов"](#).
2. PostgreSQL. [Часто Задаваемые вопросы](#)